

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ НАЦІОНАЛЬНОМУ
УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”**

Кафедра систем штучного інтелекту



ЗВІТ

з дисципліни

“Проект з створення модуля аналізу даних у інформаційній системі”

на тему: «Вдосконалення токєнізатора української мови для великих мовних
моделей»

Виконав:

студент групи ШІ-42

Мирош Андрій

Викладач:

Яковина В. С.

Львів – 2026 р.

Зміст

Зміст

Зміст.....	2
Вступ.....	4
Аналіз літературних джерел.....	5
Аналіз наявних наборів даних	7
Висновки до розділу	9
ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ	10
Інформаційне забезпечення та методи токенизації.....	10
Метрики оцінювання токенизації.....	11
Висновки до розділу	13
ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ	14
Загальна характеристика програмної реалізації.....	14
Реалізація модулів і ключові рішення.....	15
Завантаження корпусу та попередня обробка.	15
Уніфікована токенизація локальним і Nugging Face токенизаторами.	15
Обчислення метрик і збереження результатів.	15
Інтерфейс командного рядка (CLI) для тестування токенизатора	15
Відтворюваність експериментів і доступність реалізації	16
Висновки до розділу	17
ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ	18
Постановка експерименту та умови відтворюваності.....	18
Результати за token fertility	19
Результати за UNK/OOV та byte fallback.....	20
Результати round-trip fidelity та інтерпретація.....	21
Узагальнення результатів та обговорення	23
Висновки до розділу	23
ЗАГАЛЬНІ ВИСНОВКИ.....	25

Список використаних джерел	27
----------------------------------	----

Вступ

Українське NLP останніми роками демонструє помітне зростання: з’являються нові корпуси, бенчмарки та прикладні системи, а українська мова все частіше входить до фокусу досліджень із багатомовного моделювання. Зокрема, у спільноті формуються ресурси для граматичної корекції та редагування, а також публікуються узагальнюючі огляди щодо переходу української з категорії “low-resource” до “high-resource” мов [1], [2]. Паралельно з ростом доступності великих мовних моделей (LLM) актуалізується питання їхньої ефективної роботи з українським текстом, де одним із ключових вузлів у пайплайні є саме токенизація [3], [4]

Актуальність теми. Токенизація визначає спосіб перетворення тексту на дискретні індекси словника та безпосередньо впливає на довжину послідовностей у токенах, отже — на використання контекстного вікна, швидкодію та потенційні втрати якості для морфологічно багатих мов. У сучасних дослідженнях показано, що токенизаційні рішення можуть бути чутливими до мовної варіативності, домену та орфографічних відмінностей, а також по-різному впливати на поведінку моделей у різних мовах [5]. Для української мови це особливо важливо через розвинену словозміну, продуктивне словотворення та наявність специфічних орфографічних/Unicode-особливостей.

Мета роботи. Метою роботи є вдосконалення (розробка та налаштування) спеціалізованого токенизатора української мови на основі підходу subword-segmentation (BPE) та побудова відтворюваного ланцюга оцінювання його якості у порівнянні з токенизаторами сучасних LLM [3], [4].

Завдання дослідження. Для досягнення мети необхідно:

1. сформулювати вимоги до токенизації українських текстів та визначити правила попередньої нормалізації
2. реалізувати/адаптувати BPE-токенизатор та інструменти його використання
3. визначити метрики порівняння (зокрема token fertility та OOV/UNK або byte fallback) і реалізувати їх обчислення
4. провести експериментальне порівняння з токенизаторами поширених LLM та проаналізувати результати на репрезентативному корпусі [3], [4].

Об’єкт дослідження. Об’єктом дослідження є процес токенизації тексту в задачах обробки природної мови та взаємозв’язок між структурою словника, алгоритмом сегментації й статистичними характеристиками отриманих токен-послідовностей.

Предмет дослідження. Предметом дослідження є методи subword-токенизації (зокрема BPE та суміжні підходи), правила нормалізації й пре-токенизації для українських текстів, а також метрики оцінювання ефективності токенизації на корпусах [3], [4], [5].

Методи дослідження. У роботі застосовано методи статистичної обробки текстових даних, subword-моделювання словника (BPE) [7], порівняльний експеримент на корпусі та агрегування метрик (середнє, перцентилі). Для інтерпретації результатів використано підходи з сучасних робіт із оцінювання токенизаторів та аналізу їх впливу на мовні моделі [3], [5], [6].

Практичне значення. Практичним результатом є відтворюваний інструментарій для тестування токенизатора (скрипти/CLI) та отримані порівняльні оцінки, які можна використати при адаптації LLM до української мови або при виборі токенизатора для україномовних NLP-сервісів. Результати також можуть бути корисними для подальшої інтеграції українських ресурсів у багатомовні рішення [2], [7].

Аналіз літературних джерел

Токенизація є базовим етапом у більшості сучасних NLP-конвеєрів: вона задає дискретне представлення тексту через словник і визначає, як саме модель “бачить” морфеми, частини слів та символи. Класичні підходи subword-токенизації виникли як відповідь на проблему відкритого словника: замість великих списків слів модель працює з підсловами, що дозволяє кодувати рідкісні форми через композицію частин. Одним із найвідоміших методів є Byte-Pair Encoding (BPE), який ітеративно об’єднує часті пари символів/підрядків та формує словник підсловних одиниць [8]. Практика показала, що subword-підхід дає змогу зменшити кількість OOV та підвищити стабільність роботи на рідкісних словах, що особливо актуально для мов із багатою морфологією.

Подальший розвиток отримали “мовонезалежні” реалізації, зокрема SentencePiece, що дозволяє навчати subword-модель без попереднього розбиття на слова та підтримує різні схеми сегментації (включно з BPE та

unigram-LM підходом) [9]. Це важливо для відтворюваності та масштабування: єдина процедура навчання токенизатора на сирому тексті спрощує порівняння моделей і мінімізує артефакти ручної пре-токенізації, які можуть суттєво впливати на кінцевий розподіл токенів.

Однак сама по собі “компресія тексту” (коротша послідовність токенів) не є вичерпною характеристикою якості токенизатора. Сучасні роботи пропонують розглядати токенізацію як багатовимірний компроміс між компактністю, покриттям рідкісних явищ та стабільністю до варіацій написання. Зокрема, у дослідженні з масштабним порівнянням токенизаторів запропоновано системно оцінювати токенизатори на різних масштабах даних та з використанням статистичних метрик, які описують “роздування” послідовності та її розподіли (середні значення, перцентилі тощо) [3]. Окремо показано, що токенізація може бути чутливою до мовної варіативності (регістр, орфографія, стильові відмінності), що змінює сегментацію й потенційно впливає на поведінку моделі в доменах, відмінних від тренувального [5].

Для підсловних методів також відомі структурні обмеження. Наприклад, продемонстровано, що ВРЕ як жадібна процедура побудови словника не завжди оптимально відновлює морфологічно узгоджені одиниці, і в деяких сценаріях альтернативи (на кшталт unigram-LM сегментації) можуть давати кращі властивості розбиття [6]. Це підкреслює, що оцінка токенизатора має враховувати не лише середню довжину токен-послідовностей, а й поведінку на “важких” випадках (довгі, рідкісні, складні слова), що зручно відображається через високі перцентилі token fertility (наприклад, 95-й).

Окремий напрям літератури присвячений тому, як спеціалізована видозміна токенизатора допомагає “недопредставленим” мовам. Показано, що спеціалізовані або розширені словники можуть суттєво покращити ефективність представлення тексту та частково компенсувати “англоцентричність” стандартних токенизаторів, покращуючи двомовну/багатомовну підтримку без радикальної зміни архітектури моделі [7]. Для української мови це узгоджується із загальною тенденцією до накопичення ресурсів і переходу до стану, коли можливі системні покращення інфраструктурних компонентів (корпуси, токенизатори, бенчмарки), а не лише “точкові” моделі під окремі задачі [2].

У прикладній площині важливими є українські набори даних та оцінки токенизаційної ефективності на реальних корпусах. Наприклад, багатомовні ресурси для grammatical error correction з українським підкорпусом демонструють, що якісні дані існують і можуть бути використані як для тренування/адаптації моделей, так і для чесного порівняння підходів [1]. Додатково, спеціалізована робота з оцінювання ефективності токенизації для української на токенизаторах foundational-моделей підкреслює, що різні “state-of-the-art” токенизатори дають відмінні значення token fertility і по-різному поведуться на українських текстах [4]. Отже, існує практична мотивація до побудови українського токенизатора та стандартизованого, відтворюваного пайплайна оцінювання.

Таким чином, аналіз літератури вказує на три ключові висновки. По-перше, subword-токенизація (BPE та її варіанти) є де-факто стандартом, але її якість суттєво залежить від нормалізації, пре-токенизації та складу корпусу [8], [9]. По-друге, метрики на кшталт token fertility та аналіз перцентилів є практичним способом порівняння токенизаторів, проте їх потрібно інтерпретувати у зв’язці зі стабільністю до варіативності та покриттям рідкісних явищ [3], [5], [9]. По-третє, для української мови існує як теоретична, так і прикладна підстава для спеціалізації токенизатора та порівняння з токенизаторами сучасних LLM на українських корпусах [2], [4]. Саме ці положення формують основу постановки задачі та вибору метрик у даній роботі.

Аналіз наявних наборів даних

Для побудови й об’єктивного порівняння токенизаторів критично важливим є вибір корпусу, який (а) є достатньо великим, (б) охоплює різні домени та стилі мовлення, (в) репрезентує сучасну українську мову та її реальне використання. У задачі оцінювання токенизації корпус відіграє роль “вимірювального середовища”: метрики фертильності та fallback/UNK-частки безпосередньо залежать від частот морфем, доменних термінів, питомої частки іншомовних запозичень, а також від якості нормалізації символів.

У наявній інфраструктурі україномовних ресурсів можна виокремити щонайменше три класи наборів даних. Перший клас — **великі неанотовані корпуси**, які використовуються для попереднього навчання мовних моделей та/або навчання токенизаторів. Другий клас — **структурно або лінгвістично**

анотовані корпуси, що дозволяють проводити точніші експерименти на рівні морфології/синтаксису, проте зазвичай поступаються обсягом. Третій клас — **задачево-орієнтовані датасети** (наприклад, GEC), які відображають прикладні сценарії та містять складні “шумні” випадки (помилки, редагування, розмовні конструкції), важливі для оцінки поведінки токенизатора на неідеальному тексті.

У межах даної роботи для порівняння токенизаторів використовується **Kobza Corpus**, опублікований як датасет на Hugging Face та описаний у працях, присвячених підвищенню ресурсності української мови. Його перевагою є поєднання різних джерел (енциклопедичні тексти, новини, соціальні медіа тощо), що наближує розподіл даних до реальних застосувань, а також факт використання цього корпусу в українських NLP-дослідженнях як базового “широкого” джерела текстів.

Водночас, як і для багатьох великих зібраних корпусів, актуальними залишаються ризики залишкового шуму та неоднорідності, оскільки абсолютна якість кожного фрагмента складно гарантується на етапі агрегації. У твоєму тексті ці обмеження вже зафіксовані як потенційний фактор впливу на результати оцінювання.

Окремо слід підкреслити важливість **узгодженої нормалізації** перед токенизацією. Для української це включає, зокрема, NFC-нормалізацію Unicode та уніфікацію апострофів і подібних символів, які в реальних текстах часто представлені різними кодовими точками (через різні клавіатури, редактори та джерела). Запровадження такої нормалізації зменшує “штучне” зростання рідкісних символів і робить порівняння токенизаторів коректнішим, оскільки різні моделі по-різному реагують на нетипові апострофи/лапки.

Крім Kobza, для української мови існують задачево-орієнтовані та анотовані ресурси, які корисні для перевірки поведінки токенизатора в прикладних умовах. Наприклад, UA-GEC надає професійно анотований корпус виправлень граматичних та стилістичних помилок у різних доменах і може слугувати джерелом “шумних” прикладів, де токенизатор частіше стикається з нетиповими написаннями та помилками [10]. OmniGEC, у свою чергу, розширює масштаби граматичної корекції до мультимовного рівня й також містить українські дані, що дозволяє аналізувати узагальнення та перенос між мовами [1]. Додатковий приклад анотованого ресурсу — UD-деревбанк парламентських промов, який репрезентує напівспонтанне мовлення та

офіційний дискурс одночасно й може бути корисним для аналізу морфологічних форм у специфічному жанрі [11].

Таким чином, обраний підхід — використання великого різнодоменого корпусу як бази для “глобальних” метрик ефективності токенизації, доповнений перспективою перевірки на задачево-орієнтованих/анотованих наборах даних — є методологічно виправданим. Він дозволяє оцінити токенизатор одночасно як компонент для довгих реальних текстів (де важлива фертильність), так і як компонент для “складних” прикладних сценаріїв (де важливі fallback/UNK та round-trip).

Висновки до розділу

У розділі проаналізовано наукові підходи до субслівної токенизації та причини, через які ефективність токенизатора має критичне значення для сучасних трансформерних моделей. Показано, що класичні алгоритми BPE, WordPiece та SentencePiece забезпечують відкритий словник і сумісність із нейромережовим моделюванням, однак їхня практична ефективність для конкретної мови залежить від розподілу даних, політики fallback-кодування та чутливості до варіативності написання. Для української мови, яка характеризується морфологічною складністю та історичною нерівномірністю ресурсів, ця проблема проявляється особливо відчутно, що підтверджується сучасними роботами з аналізу токенизації для українських корпусів [2], [4].

Також обґрунтовано вибір корпусу для порівняння токенизаторів: використання Kobza як різнодоменого джерела дозволяє вимірювати усереднену ефективність токенизації на матеріалі, близькому до практичних застосувань, за умови узгодженої нормалізації тексту. Окреслено, що додаткові набори даних (на кшталт GEC та UD-корпусів) можуть бути корисними для подальших перевірок поведінки токенизатора на неідеальному тексті та специфічних жанрах. На основі цього сформовано підґрунтя для наступних розділів роботи, у яких описується реалізація токенизатора та експериментальна методологія оцінювання за метриками фертильності, UNK/byte fallback та round-trip fidelity.

ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

Інформаційне забезпечення та методи токенизації

Інформаційною основою експериментів у даній роботі є україномовний корпус текстів, який використовується як спільне середовище для порівняння різних токенизаторів. Корпус подається на вхід усім токенизаторам у стандартизованому вигляді, що дозволяє порівнювати результати за принципом “за інших рівних умов”. Важливою вимогою до даних є наявність природної лексичної та стилістичної варіативності (морфологічні форми, різні жанри, іншомовні вставки), оскільки саме ці фактори найбільше впливають на токенизаційну ефективність.

Перед токенизацією застосовується єдина процедура попередньої обробки, спрямована на усунення артефактів запису тексту. По-перше, текст нормалізується до канонічної форми Unicode NFC, що усуває різні способи подання однакових символів. По-друге, виконується уніфікація апострофів і подібних символів до стандартного ASCII-апострофа, оскільки в українських текстах різні варіанти апострофа часто виникають через різні джерела даних та редактори. Така нормалізація зменшує штучну фрагментацію токенів і робить порівняння токенизаторів коректнішим.

Математично токенизація розглядається як відображення з множини рядків у множину послідовностей цілих індексів словника. Для підсловникових підходів (subword) текст розбивається на підрядки, які належать словнику токенизатора. У роботі фокус зроблено на BPE-підході та порівнянні з токенизаторами сучасних LLM, які зазвичай реалізують одну з форм субслівної токенизації або її модифікації. Базова мотивація BPE полягає в тому, що замість жорсткого словника повних слів використовується словник субодиноць, який дозволяє кодувати рідкісні або нові слова через композицію частин [8]. Альтернативний практичний стандарт — SentencePiece, який надає мовнезалежний механізм навчання й застосування subword-токенизатора та формалізує процес детокенизації (відновлення рядка) [9]. У багатьох сучасних системах також використовується WordPiece-подібна сегментація, зокрема в архітектурах BERT-сімейства [6]. Окремо у літературі підкреслюється, що BPE може бути не оптимальним у задачах переднавчання

мовних моделей, а вибір схеми сегментації впливає на властивості послідовностей і статистику “роздування” тексту [3].

Для цілей цієї роботи ключовим є не внутрішня оптимізаційна процедура побудови словника, а емпіричні властивості отриманих токен-послідовностей на українському корпусі, тому порівняння здійснюється через метрики ефективності токенизації та коректності перетворень, описані в наступному підрозділі.

Метрики оцінювання токенизації

Оцінювання якості токенизаторів у роботі базується на метриках, які характеризують ефективність представлення українського тексту в токенах, ступінь покриття рідкісних випадків та зворотність перетворення “текст $\leftrightarrow$ токени”. Обрані метрики не потребують запуску або навчання повноцінної мовної моделі, що забезпечує відтворюваність і зменшує обчислювальні витрати при порівнянні великої кількості токенизаторів.

Першою основною метрикою є **середня кількість токенів на слово (token fertility)**. Для її обчислення текст розбивається на слова за пробілами (whitespace segmentation), після чого кожне слово токенизується окремо. Нехай корпус містить N слів $w_1, \dots, w_N$, а $|T(w_i)|$ — кількість токенів, отриманих при токенизації слова w_i . Тоді середня фертильність визначається як:

$$F_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N |T(w_i)|.$$

Окрім середнього значення, аналізуються перцентилі розподілу $|T(w_i)|$, зокрема 95-й перцентиль F_{p95} , який відображає “погану” поведінку токенизатора на складних, довгих або рідкісних словах. Інтерпретаційно менші значення F_{mean} і F_{p95} означають компактніші послідовності, а отже ефективніше використання контекстного вікна LLM.

Другою основною метрикою є **частка OOV/UNK або частка byte fallback**. Вона відображає, яка частина токенизованого тексту кодується не “нормальними” одиницями словника, а спеціальними механізмами для

непокритих випадків. Для токенизаторів із явним UNK-токеном частка визначається як:

$$U = \frac{\#UNK}{\#Tokens},$$

де $\#UNK$ — кількість tokenів типу UNK у корпусі, а $\#Tokens$ — загальна кількість tokenів після токенизації текстів. Для байтових/байт-фолбек токенизаторів UNK-частка часто дорівнює нулю, тому змістовнішою стає **частка byte fallback**, тобто відношення кількості tokenів, що представляють “сирі” байти, до загальної кількості tokenів. У літературі така властивість розглядається як важливий механізм забезпечення повного покриття Unicode/нетипових символів, проте надмірна частка fallback може свідчити про слабе покриття словником саме для цільової мови.

Третьою метрикою (валідаційною) є **точність зворотного перетворення (round-trip fidelity)**. Вона перевіряє, чи відновлюється рядок після операцій токенизації та декодування без втрат інформації, з урахуванням узгодженої нормалізації. Нехай x — вхідний рядок, $\text{norm}(\cdot)$ — процедура нормалізації (у роботі це NFC та уніфікація апострофів), тоді round-trip збіг фіксується тоді, коли:

$$\text{norm}(\text{decode}(\text{encode}(x))) = \text{norm}(x).$$

Частка точних збігів на наборі тестових рядків $x_1, \dots, x_M$ визначається як:

$$R = \frac{1}{M} \sum_{j=1}^M \mathbb{I} [\text{norm}(\text{decode}(\text{encode}(x_j))) = \text{norm}(x_j)].$$

Значення R , близьке до 1, означає коректну зворотність і відсутність втрат при encode/decode. Значення, менші за 1, сигналізують про незворотні перетворення (наприклад, агресивну нормалізацію або проблеми з пробілами/Unicode), що може бути критичним у прикладних сценаріях, де важливо точно відтворювати текст.

Таким чином, набір метрик $F_{\text{mean}}, F_{p95}, U$ (або byte fallback) та R дозволяє комплексно оцінювати токенизатор у трьох площинах: ефективність (довжина

послідовностей), покриття (обробка рідкісних випадків), та коректність перетворення (зворотність).

Висновки до розділу

У розділі визначено інформаційні передумови та математичний апарат, необхідний для коректного порівняння токенізаторів на українських текстах. Показано, що для відтворюваного експерименту критично важливими є узгоджені правила нормалізації (NFC та уніфікація апострофів) і єдина процедура сегментації на слова, оскільки вони безпосередньо впливають на статистику токенізації. Також обґрунтовано вибір метрик, які дозволяють оцінювати токенізатор без навчання мовної моделі: token fertility (середнє та перцентилі) як показник компактності представлення, частка UNK/byte fallback як показник покриття рідкісних явищ, та round-trip fidelity як валідаційний критерій відсутності втрат інформації при encode/decode. Обраний набір метрик формує основу для реалізації експериментального конвеєру та інтерпретації результатів у наступному розділі.

ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ

Загальна характеристика програмної реалізації

Для розв’язання задачі порівняння токенизаторів було реалізовано мінімально працездатну систему (MVP), призначену для завантаження корпусу, уніфікованої попередньої обробки текстів, токенизації різними токенизаторами, обчислення метрик та збереження результатів у формі, зручній для подальшого аналізу. Система побудована як набір модулів, що відповідають окремим етапам експериментального конвеєру, що спрощує розширення та повторне використання коду.

Важливим принципом реалізації є відокремлення оцінювання токенизатора від запуску повноцінної мовної моделі. Усі порівняння проводяться в режимі “текст → токени” (і, за потреби, “токени → текст”), що дозволяє виконувати експерименти без значних обчислювальних ресурсів та забезпечує відтворюваність.

3.2 Архітектура MVP-системи

Архітектурно система складається з трьох основних компонентів:

1. **Модуль обробки даних:** відповідає за завантаження корпусу, нормалізацію та сегментацію. Нормалізація включає приведення тексту до Unicode NFC та уніфікацію апострофів; сегментація слів виконується за пробілами для коректного вимірювання “токенів на слово”.
2. **Модуль токенизації:** підтримує як локальний BPE-токенизатор, так і токенизатори LLM, доступні через Hugging Face. Для всіх токенизаторів застосовується уніфікований інтерфейс виклику, а під час токенизації не додаються службові токени, які могли б спотворювати статистику довжин послідовностей.
3. **Модуль обчислення метрик та агрегації результатів:** автоматично обробляє весь корпус для кожного токенизатора, обчислює задані метрики, агрегує їх та зберігає результати у структурованому форматі для побудови графіків і підготовки звіту.

Додатково реалізовано **інтерфейс командного рядка (CLI)**, який дозволяє користувачу тестувати токенизацію на довільних рядках та запускати частину процедур без прямої взаємодії з кодом.

Реалізація модулів і ключові рішення

Завантаження корпусу та попередня обробка.

У роботі використано корпус Kobza, доступний на платформі Hugging Face. Завантаження виконується у потоковому режимі (streaming), що дозволяє обробляти корпус без повного завантаження в оперативну пам'ять і працювати з фіксованою кількістю зразків для відтворюваного порівняння.

Перед токенизацією тексти проходять єдину нормалізацію. Це зменшує вплив “технічних” відмінностей у поданні символів (Unicode-композиція) та усуває варіативність апострофа, яка особливо поширена в українських даних з різних джерел. Додатково застосовується однакове правило сегментації слів (whitespace), що забезпечує коректність і порівнюваність метрики token fertility.

Уніфікована токенизація локальним і Hugging Face токенизаторами.

Ключова вимога до модулю токенизації — забезпечити однаковий спосіб отримання токенів для різних реалізацій токенизаторів. Система підтримує локальний ВРЕ-токенизатор української мови та токенизатори сучасних великих мовних моделей, а для справедливого порівняння застосовує єдиний інтерфейс виклику. Також спеціально контролюється, щоб під час токенизації не додавалися службові токени (BOS/EOS тощо), які могли б штучно збільшувати довжину послідовності.

Обчислення метрик і збереження результатів.

Оцінка якості токенизації виконується окремим модулем, який проганяє корпус для кожного токенизатора та формує агреговані метрики. Результати зберігаються у структурованому форматі, що спрощує подальшу візуалізацію та інтеграцію у звіт.

Інтерфейс командного рядка (CLI) для тестування токенизатора

Для взаємодії з розробленим токенизатором та інструментами оцінки було реалізовано інтерфейс командного рядка (Command Line Interface, CLI). Даний інтерфейс дозволяє користувачеві виконувати токенизацію довільних

текстових рядків(Фіг. 2), отримувати базову статистику токенизації, а також запускати обчислення метрик на корпусі даних без необхідності безпосередньої взаємодії з кодом.

CLI-інтерфейс підтримує вбудовану довідку(Фіг. 1), що містить опис доступних команд і параметрів запуску. Це спрощує використання інструменту та робить його придатним для практичного застосування іншими користувачами. На рисунку наведено приклад виклику довідкової інформації та приклад токенизації українського тексту.

Реалізація CLI підтверджує, що розроблена система є не лише аналітичним прототипом, а повноцінним мінімально працездатним програмним продуктом, який може бути використаний для тестування та аналізу токенизації українських текстів у прикладних сценаріях.

```
Usage: ukr_tokenizer_cli.py [OPTIONS] COMMAND [ARGS]...

Ukrainian tokenizer CLI: test tokenization, inspect tokens, and compute basic metrics.

Options
  --install-completion  Install completion for the current shell.
  --show-completion     Show completion for the current shell, to copy it or customize the installation.
  --help               Show this message and exit.

Commands
  tokenize  Tokenize a single piece of text and print tokenization details.
  decode    Decode token IDs back to text (if supported by the local tokenizer).
  repl     Interactive prompt: type text and see tokenization; type /quit to exit.
  stats    Compute basic corpus stats: tokens/word and tokens/char on a local file or stdin.
  export   Tokenize each line and write JSONL with {text, ids, tokens, token_count, word_count}.
```

Фіг. 1. Демонстрація меню довідки

```
(.venv) C:\Users\Andrii\My projects\ukr-tokenizer>python ukr_tokenizer_cli.py tokenize "Привіт, як справи?"
None of PyTorch, TensorFlow >= 2.0, or Flax have been found. Models won't be available and only tokenizers,
Loaded tokenizer from: ukr_bpe_tokenizer\tokenizer.json
Vocabulary size: 32,000
Tokenizer: Local BPE Tokenizer
Chars: 18 Words: 3 Tokens: 6
Fertility (tok/word): 2.000
Tokens/char: 0.3333 Chars/token: 3.000

Token IDs:
[12691, 12367, 41, 12162, 14510, 60]

Tokens:
['_При', 'віт', ' ', ' ', '_як', '_справи', '?']
```

Фіг. 2. Демонстрація роботи

Відтворюваність експериментів і доступність реалізації

Побудований конвеєр забезпечує відтворюваність порівняння: усі токенизатори оцінюються на одному й тому самому корпусі після ідентичної попередньої обробки, з однаковим набором метрик та без обмеження максимальної довжини послідовностей, щоб уникати спотворення статистики.

Уся реалізація токенизатора, інструментів оцінки та експериментального порівняння розміщена у відкритому репозиторії, який містить вихідний код, інструкції з запуску та приклади використання CLI. Посилання на репозиторій: <https://github.com/AwfulIceCream/ukr-tokenizer>.

Висновки до розділу

У розділі описано програмну реалізацію системи оцінювання токенизаторів української мови. Показано, що MVP-підхід із модульною архітектурою дозволяє відокремити завантаження корпусу та нормалізацію, уніфікувати інтерфейс токенизації для локального ВРЕ та токенизаторів LLM, а також автоматизувати підрахунок метрик і збереження результатів у формі, придатній для подальшої візуалізації. Реалізований CLI додає практичний рівень взаємодії, полегшуючи тестування токенизатора та запуск основних процедур оцінювання без прямої роботи з кодом.

ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

Постановка експерименту та умови відтворюваності

Експериментальна частина роботи спрямована на кількісне порівняння ефективності українського BPE-токенізатора з токенізаторами сучасних LLM на однаковому корпусі та за узгодженими правилами попередньої обробки. Для оцінювання використовувався україномовний корпус Kobza; у проведеному прогоні було взято 10 000 текстових зразків у потоковому режимі завантаження. Перед токенізацією до всіх текстів застосовувалася єдина процедура нормалізації: Unicode NFC та уніфікація апострофів до стандартного ASCII-символу. Сегментація “слів” для метрики token fertility виконувалась за пробілами (whitespace).

Порівняння проводилось для таких токенізаторів:

1. Local BPE Tokenizer (український токенізатор, навчений на українських даних);
2. CohereForAI/aya-expanse-8b;
3. Qwen/Qwen2.5-7B-Instruct;
4. google/gemma-3-12b-it;
5. microsoft/Phi-4-mini-instruct;
6. meta-llama/Llama-3.1-8B-Instruct.

Обчислювалися метрики, визначені в розділі 2:

1. середня token fertility (токенів/слово) та 95-й перцентиль;
2. частка UNK серед усіх токенів;
3. частка byte fallback серед усіх токенів (для токенізаторів, що мають явні byte-токени);

4. round-trip fidelity як частка точних збігів $\text{preprocess}(\text{decode}(\text{encode}(x))) = \text{preprocess}(x)$ на підмножині рядків.

Під час запусків для деяких Hugging Face токенизаторів з'являлися попередження про те, що довжина послідовності токенів перевищує `model_max_length`. Оскільки в експериментах не виконувався прогін моделі (лише токенизація), ці попередження не впливають на значення метрик, але їх варто враховувати при інтеграції токенизатора в модельний інференс (де вже потрібне керування `truncation`/контекстним вікном).

Результати за token fertility

На Рисунку 1 наведено середню token fertility (токенів на слово), обчислену на 100000 зразках корпусу. Український Local BPE Tokenizer показав найкращий результат: **1.803 токена/слово**, що відповідає цільовому орієнтиру < 2 токени/слово. Найкращий із порівнюваних “універсальних” токенизаторів (CohereForAI/aya-expanse-8b) має **2.554 токена/слово**, тоді як інші — **2.698** (google/gemma-3-12b-it), **2.852** (meta-llama/Llama-3.1-8B-Instruct), **2.862** (microsoft/Phi-4-mini-instruct) та **3.613** (Qwen/Qwen2.5-7B-Instruct).



Рис 1. Середнє значення фертильності

Таким чином, у середньому Local BPE Tokenizer зменшує довжину токенизованої послідовності приблизно на **29%** відносно найкращого базового рівня LLM (aya-expanse-8b) та до **50%** відносно найгіршого в цьому порівнянні (Qwen2.5-7B-Instruct). Практично це означає, що за фіксованого контекстного вікна LLM український токенизатор дозволяє помістити суттєво

більше “змістових” слів у той самий ліміт токенів, що напряду покращує придатність до довгих документів і діалогів.

Додатково важливо аналізувати “хвіст” розподілу — 95-й перцентиль token fertility (Рисунок 2). Для Local BPE Tokenizer **p95 = 4 токени/слово**, тоді як для більшості порівнюваних LLM-токенізаторів **p95 = 5**, а для Qwen2.5-7B-Instruct **p95 = 7**. Різниця у p95 демонструє, що локальний токенізатор не лише зменшує середню довжину, а й краще поводить себе на “складних” словах (довгі словоформи, рідкісні лексеми, похідні та складені слова). Зменшення p95 з 5 до 4 відповідає приблизно **20%** скороченню токенів у верхньому хвості розподілу, що особливо критично для морфологічно багатих мов, де саме рідкісні словоформи часто спричиняють надмірне дроблення.

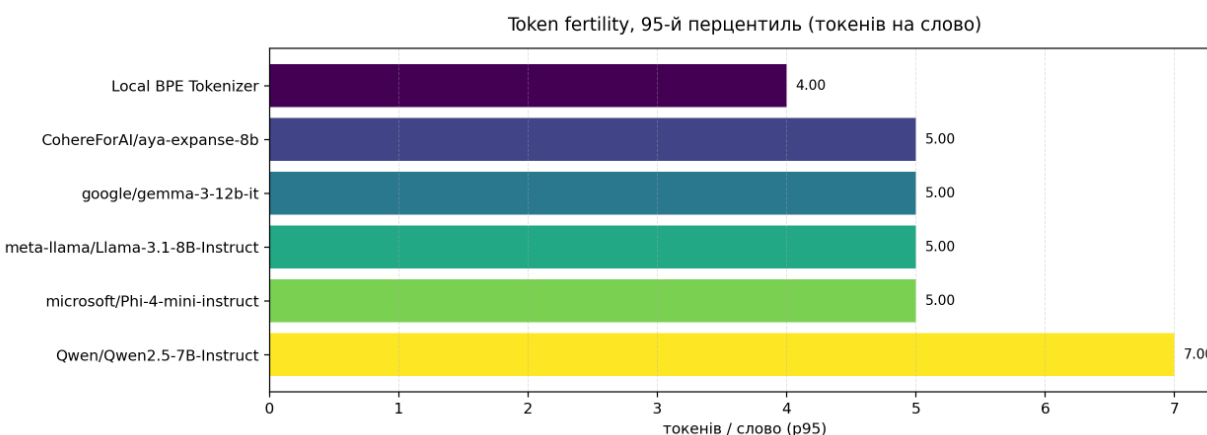


Рис. 2. 95-ий перцентиль фертильності.

Результати за UNK/OOV та byte fallback

На Рисунку 3 наведено частку UNK серед усіх токенів. У проведеному експерименті **в усіх токенізаторів UNK-частка дорівнює 0%**. Це узгоджується з тим, що багато сучасних токенізаторів реалізують байтове покриття (byte-level) або механізми, які дозволяють уникати явного UNK для довільного тексту. Водночас, нульове значення UNK не означає “ідеального покриття словником”: частина символів/фрагментів може кодуватися як байтові токени або дуже дрібні субодиниці, що збільшує token fertility.

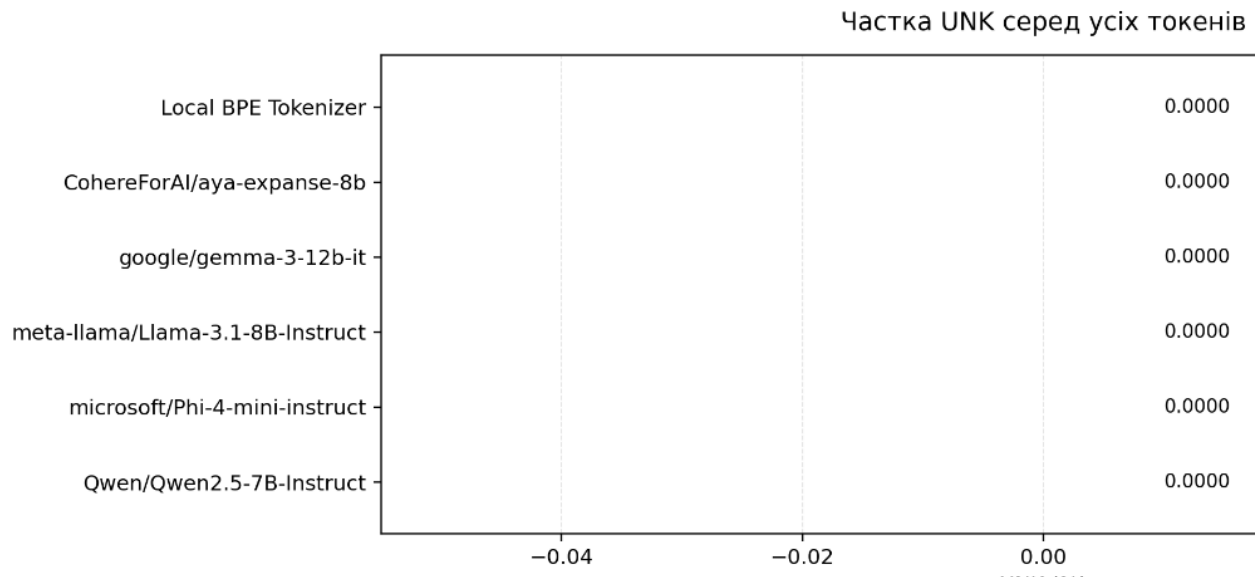


Рис. 3. Частка невідомих токенів

Саме тому додатково було розраховано частку byte fallback (Рисунок 4) для токенізаторів, у словнику яких присутні явні byte-токени (у форматі на кшталт <0xAB>). У цих умовах byte fallback було зафіксовано для **google/gemma-3-12b-it** на рівні **0.0318%** від усіх токенів, що є дуже малим значенням і свідчить про поодинокі випадки переходу до байтового подання (наприклад, рідкісні символи, технічні маркери або нетипові Unicode-послідовності в корпусі). Для інших токенізаторів показник не відображався як “n/a”, що інтерпретується як відсутність явних byte-токенів у словнику або відмінний механізм fallback, який потребує окремої ідентифікації.

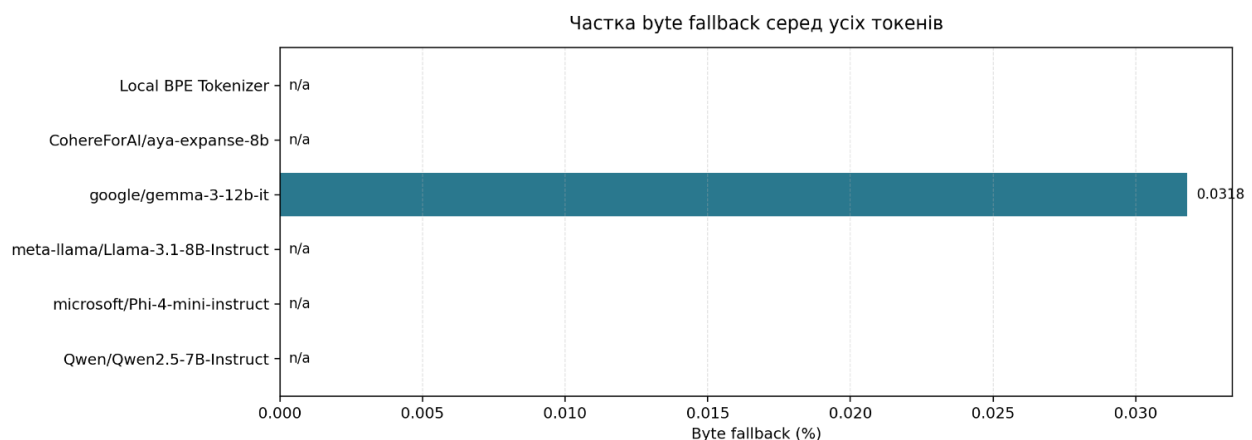


Рис 4. Частка byte fallback

Результати round-trip fidelity та інтерпретація

Round-trip fidelity (Рисунок 5) перевіряє, чи зберігається текст після перетворення $\text{encode} \rightarrow \text{decode}$ за узгодженої нормалізації.

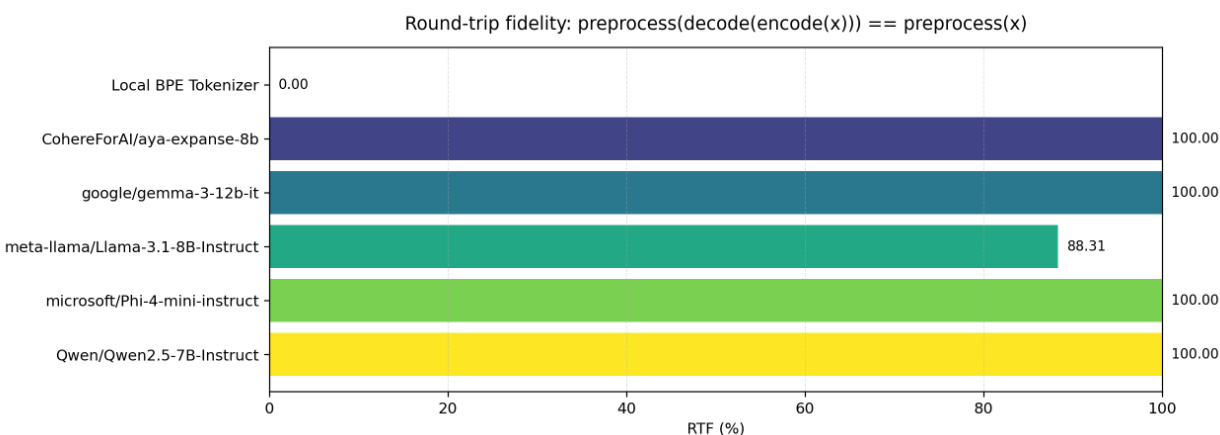


Рис. 5. Значення round-trip fidelity

У результатах спостерігаються три характерні ситуації:

1. **100% round-trip fidelity** для CohereForAI/aya-expanse-8b, google/gemma-3-12b-it, microsoft/Phi-4-mini-instruct та Qwen/Qwen2.5-7B-Instruct. Це означає, що для вибірки текстів, використаної у перевірці, ці токенизатори відтворюють рядок без втрат (з урахуванням попередньої нормалізації).
2. **88.31% round-trip fidelity** для meta-llama/Llama-3.1-8B-Instruct. Такий результат найчастіше пов'язаний не з “втратою змісту”, а з дрібними відмінностями у пробілах/керівних символах або політикою роботи з початковим пробілом (prefix space). Частина токенизаторів під час кодування/декодування може додавати або інтерпретувати пробіл на межі рядка, а порівняння у метриці є строгим. Практичний висновок: для цього токенизатора потрібно або уточнити політику порівняння (наприклад, фіксувати правило щодо початкових/кінцевих пробілів), або ж налаштувати режим токенизації так, щоб round-trip був строгим у прикладних сценаріях, де важлива буквальна тотожність рядка.
3. **0% round-trip fidelity** для Local BPE Tokenizer. Такий результат, як правило, означає технічну проблему з детокенізацією: у локальному tokenizer.json може бути відсутньо або некоректно налаштовано декодер (decoder), або ж токенизатор використовує внутрішні маркери, які не відновлюються назад у вихідний рядок стандартною процедурою

decode. Важливо підкреслити, що це не обов’язково знецінює token fertility-результати (бо вони базуються на encode), але вказує на необхідність допрацювання токенизатора як “повного” перетворювача тексту (з коректною операцією decode). Для практичного використання в ланцюгах обробки, де потрібне відновлення тексту (наприклад, для дебагу, аналізу, інтеграції зі сторонніми системами), виправлення round-trip є обов’язковим кроком.

Узагальнення результатів та обговорення

Експеримент показує, що спеціалізований український токенизатор дає відчутний вииграш за ключовою прикладною метрикою — token fertility. З огляду на те, що token fertility безпосередньо впливає на те, скільки тексту можна вмістити у контекстне вікно та які будуть обчислювальні витрати, отримане значення 1.803 токена/слово є важливим практичним результатом.

Водночас результати також підсвічують компроміси. По-перше, нульова UNK-частка у сучасних LLM-токенизаторів може співіснувати з високою token fertility: відсутність UNK досягається завдяки fallback-механізмам, але текст може кодуватися довгими послідовностями дрібних одиниць. По-друге, для локального токенизатора необхідно забезпечити коректний decode, оскільки round-trip fidelity — це базова властивість “інформаційної зворотності”, важлива для багатьох прикладних сценаріїв. По-третє, round-trip проблеми на рівні 88.31% у Llama-3.1 вказують, що навіть у відомих токенизаторів можуть бути нюанси з пробілами/межами рядка, які слід явно нормалізувати або описувати як частину політики експерименту.

Висновки до розділу

1. Український Local BPE Tokenizer продемонстрував найкращу ефективність кодування українського тексту: середня token fertility становить 1.803 токена/слово, що нижче за всі порівнювані LLM-токенизатори та відповідає практичній цілі “<2 токени/слово”.
2. Перевага зберігається і на складних випадках: p95 token fertility для локального токенизатора дорівнює 4 токени/слово, тоді як для більшості LLM-токенизаторів p95 = 5, а для Qwen2.5-7B-Instruct p95 = 7.
3. Частка UNK зафіксована як 0% для всіх токенизаторів у даному експерименті, що узгоджується з використанням сучасних механізмів

покриття тексту без явного UNK. Окремо зафіксовано низьку частку byte fallback для gemma-3-12b-it (0.0318%), що відповідає поодиноким fallback-випадкам.

4. Round-trip fidelity виявила важливі технічні моменти: 100% для більшості токенизаторів, 88.31% для Llama-3.1 (ймовірно через обробку пробілів/меж рядка) та 0% для локального токенизатора (ознака відсутнього/некоректного декодера). Це формує чіткий напрям подальшого покращення локального токенизатора: забезпечити коректний decode та повторно перевірити round-trip.

ЗАГАЛЬНІ ВИСНОВКИ

У даній роботі було розроблено та експериментально оцінено спеціалізований ВРЕ-токенізатор для української мови, а також побудовано відтворюваний інструментарій для порівняння токенізаторів сучасних великих мовних моделей на україномовному корпусі. Основний акцент роботи зроблено на метриках, які дозволяють оцінювати властивості токенізації без навчання повноцінної мовної моделі, тобто в режимі “текст → токени” (та, за потреби, “токени → текст”). Це забезпечило практичну відтворюваність експериментів і дозволило порівняти декілька LLM-токенізаторів у єдиних умовах.

1. Реалізовано мінімально працездатну систему (MVP) для оцінювання токенізаторів, яка включає: завантаження корпусу (Kobza) у потоковому режимі, уніфіковану попередню обробку тексту (Unicode NFC та уніфікація апострофів), уніфікований інтерфейс токенізації для локального та Hugging Face токенізаторів, обчислення метрик і збереження результатів у структурованому форматі. Додатково реалізовано CLI-інструмент для тестування токенізації на довільних рядках та запуску оцінювання без прямої взаємодії з кодом.
2. Запропоновано та використано набір метрик оцінювання токенізаторів, релевантний до практичних потреб LLM: token fertility (середнє та високі перцентилі як міра “роздування” послідовності), частка UNK/OOV або частка byte fallback (як показник непокриття словником/перехід до байтового кодування), а також round-trip fidelity як валідаційний критерій коректності операцій encode/decode за узгодженої нормалізації.
3. Експериментальне порівняння на 100000 зразках корпусу Kobza показало, що локальний український ВРЕ-токенізатор забезпечує найменшу середню token fertility серед розглянутих токенізаторів: 1.803 токена/слово. Для порівняння, найкращий серед LLM-токенізаторів у цьому прогоні (aya-expanse-8b) має 2.554 токена/слово, тоді як інші токенізатори демонструють 2.698–3.613 токена/слово. Практичний висновок полягає в тому, що спеціалізований токенізатор дозволяє ефективніше використовувати контекстне вікно LLM і потенційно

зменшує обчислювальні витрати при роботі з довгими українськими текстами.

4. Аналіз 95-го перцентилу token fertility підтвердив стабільну перевагу локального токенизатора на “складних” словах: p_{95} дорівнює 4 токени/слово, тоді як для більшості LLM-токенизаторів $p_{95} = 5$, а для Qwen2.5-7B-Instruct $p_{95} = 7$. Це вказує на кращу поведінку локального токенизатора у випадках довгих або рідкісних словоформ, що є типовим явищем для української мови з багатою морфологією.
5. Частка UNK у проведеному експерименті зафіксована як 0% для всіх токенизаторів, що узгоджується з сучасними механізмами покриття тексту без явного UNK. Водночас це не означає однакової якості покриття: у деяких токенизаторів можуть використовуватися байтові або дрібні fallback-одиниці, що збільшує довжину послідовностей. У даному прогоні зафіксовано малу частку byte fallback для gemma-3-12b-it (0.0318%), що свідчить про поодинокі випадки байтового представлення.
6. Перевірка round-trip fidelity продемонструвала, що більшість LLM-токенизаторів забезпечують повну зворотність перетворення (100%), тоді як для Llama-3.1-8B-Instruct зафіксовано 88.31% (ймовірно через політику обробки пробілів/меж рядка), а для локального токенизатора — 0%, що вказує на необхідність доопрацювання компонента декодування (decoder) у локальному tokenizer.json. Таким чином, з точки зору практичного використання локального токенизатора як повноцінного перетворювача “текст ↔ токени”, ключовим напрямом покращення є реалізація або коректне налаштування decode-процедури та повторна валідація round-trip.

У підсумку, робота підтвердила доцільність створення українсько-орієнтованого токенизатора: спеціалізація на українських текстах дозволяє суттєво зменшити token fertility порівняно з універсальними токенизаторами великих моделей, що є критичним для сценаріїв із довгими документами та обмеженим контекстним вікном. Разом із тим, результати також визначили подальші кроки розвитку — забезпечення повної зворотності encode/decode для локального токенизатора та розширення валідаційних перевірок на додаткові домени та “шумні” корпуси (наприклад, GEC-набори).

Список використаних джерел

- [1] R. Kovalchuk, M. Romanyshyn, i P. Ivaniuk, «Introducing OmniGEC: A Silver Multilingual Dataset for Grammatical Error Correction», в *Proceedings of the Fourth Ukrainian Natural Language Processing Workshop (UNLP 2025)*, Vienna, Austria (online): Association for Computational Linguistics, 2025, с. 162–178. doi: 10.18653/v1/2025.unlp-1.17.
- [2] M. Haltiuk i A. Smywiński-Pohl, «On the Path to Make Ukrainian a High-Resource Language», в *Proceedings of the Fourth Ukrainian Natural Language Processing Workshop (UNLP 2025)*, Vienna, Austria (online): Association for Computational Linguistics, 2025, с. 120–130. doi: 10.18653/v1/2025.unlp-1.14.
- [3] J. F. Lotz, A. V. Lopes, S. Peitz, H. Setiawan, i L. Emili, «Beyond Text Compression: Evaluating Tokenizers Across Scales», в *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, с. 32155–32173. doi: 10.18653/v1/2025.acl-long.1546.
- [4] D. Maksymenko i O. Turuta, «Tokenization efficiency of current foundational large language models for the Ukrainian language», *Front. Artif. Intell.*, вип. 8, с. 1538165, Сep 2025, doi: 10.3389/frai.2025.1538165.
- [5] A. Wegmann, D. Nguyen, i D. Jurgens, «Tokenization is Sensitive to Language Variation», 2025, *arXiv*. doi: 10.48550/ARXIV.2502.15343.
- [6] K. Bostrom i G. Durrett, «Byte Pair Encoding is Suboptimal for Language Model Pretraining», в *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online: Association for Computational Linguistics, 2020, с. 4617–4624. doi: 10.18653/v1/2020.findings-emnlp.414.
- [7] A. Kiulian *et al.*, «From English-Centric to Effective Bilingual: LLMs with Custom Tokenizers for Underrepresented Languages», в *Proceedings of the Fourth Ukrainian Natural Language Processing Workshop (UNLP 2025)*, Vienna, Austria (online): Association for Computational Linguistics, 2025, с. 1–13. doi: 10.18653/v1/2025.unlp-1.1.
- [8] R. Sennrich, B. Haddow, i A. Birch, «Neural Machine Translation of Rare Words with Subword Units», в *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany: Association for Computational Linguistics, 2016, с. 1715–1725. doi: 10.18653/v1/P16-1162.
- [9] T. Kudo i J. Richardson, «SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing», в *Proceedings of the 2018 Conference on Empirical Methods in Natural Language*

- Processing: System Demonstrations*, Brussels, Belgium: Association for Computational Linguistics, 2018, c. 66–71. doi: 10.18653/v1/D18-2012.
- [10] O. Syvokon, O. Nahorna, P. Kuchmiichuk, i N. Osidach, «UA-GEC: Grammatical Error Correction and Fluency Corpus for the Ukrainian Language», в *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, Dubrovnik, Croatia: Association for Computational Linguistics, 2023, c. 96–102. doi: 10.18653/v1/2023.unlp-1.12.
- [11] M. Shvedova, A. Lukashevskyi, i A. Rysin, «Developing a Universal Dependencies Treebank for Ukrainian Parliamentary Speech», в *Proceedings of the Fourth Ukrainian Natural Language Processing Workshop (UNLP 2025)*, Vienna, Austria (online): Association for Computational Linguistics, 2025, c. 55–63. doi: 10.18653/v1/2025.unlp-1.7.